

OutlierCIM: Outlier-Aware Digital CIM-Based LLM Accelerator with Hybrid-Strategy Quantization and Unified FP-INT Computation

Zihan Zou¹, Shikuang Chen¹, Chen Zhang¹, Xing Wang¹, Zhichao Liu¹, Haoran Du¹,
Xin Si^{1,2}, Hao Cai^{1,2}, Bo Liu^{1,2*}

¹School of Integrated Circuits, Southeast University, Nanjing, China

²National Center of Technology Innovation for EDA, Nanjing, China

Abstract—Activation outliers in Large Language Models (LLMs), which exhibit large magnitudes but small quantities, significantly affect model performance and pose challenges for the acceleration of LLMs. To address this bottleneck, researchers have proposed several co-design frameworks with outlier-aware algorithms and dedicated hardware. However, they face challenges balancing model accuracy with hardware efficiency when accelerating LLMs in a low bit-width manner. To this end, we propose OutlierCIM, the first algorithm and hardware co-design framework for the compute-in-memory (CIM) accelerator with outlier-aware quantization algorithm. The key contributions of OutlierCIM are 1) an outlier-clustered tiling strategy that regulates memory access and reduces inefficient workloads which are both introduced by outliers, 2) a hybrid-strategy quantization and a reconfigurable double-bit CIM macro array that overcome the low storage utilization and high latency of outlier-based LLM quantization, and 3) a quantization factor post-processing strategy and a dedicated quantizer that efficiently unify the multiplication and accumulation of outlier-caused FP-INT workloads. Implemented in a 28nm CMOS technology, OutlierCIM occupies an area of 2.25 mm². When evaluated at comprehensive benchmarks, OutlierCIM achieves up to 4.54× energy efficiency improvement and 3.91× speedup compared to the state-of-the-art outlier-aware accelerators.

Index Terms—large language model, compute-in-memory, neural network accelerator, activation outlier, quantization.

I. INTRODUCTION

Large Language Models (LLMs), which are scaled from pre-trained language models [1], [2], have demonstrated exceptional performance across various natural language processing tasks, including machine translation [3], text summarization [4], and question answering [5]. Despite these advancements, LLMs' increasing size and complexity pose significant challenges to their application and deployment. For instance, the LLaMA-65B model, containing 65 billion parameters, necessitates 260 GB (twelve RTX 3090Ti GPUs) of VRAM for inference.

Quantization reduces LLM size by converting parameters to low bit-width, thereby improving model inference efficiency and memory utilization [6]–[9]. For example, the weight parameters of the LLaMA-65B model can be compressed to 4-bit integer precision, reducing the model size to 32.5 GB [11]. However, the activations of LLMs are challenging to

quantize to low bit-width due to outliers which exhibit small quantities but large magnitudes. These outliers introduce large data distribution, resulting in limited quantization bit-width.

Considering the bottleneck caused by activation outliers, several algorithms have been proposed to smooth the distribution of activations [6]–[9]. However, these algorithm-only methods introduce additional mathematical space transformations that can not be fully leveraged by general computing platforms, creating a mismatch between hardware and algorithm. As a result, hardware and algorithm co-design frameworks have been developed to enhance the benefits of quantization [12]–[15]. These frameworks quantize activation outliers with high data precision and design dedicated hardware accelerators. However, the balance of hardware and algorithm is still not optimally achieved. GOBO [12] and OLAcel [13] treat outliers as sparse matrices which require additional complex control logic. OliVe [14] eliminates neighboring values of outliers to enable localized outlier storage, resulting in significant model accuracy loss at low bit-width quantization. Oltron [15] increases data precision of normal values in outlier channels and permutes channels to maintain regular bit-width for each input tiling block. However, it undergoes low efficiency at nonlinear workloads. Furthermore, the hardware implementations of these approaches face a memory wall challenge due to the memory-bound characteristic of LLMs [11], creating an additional bottleneck.

In this paper, we propose OutlierCIM to address the above challenges. The contributions are listed as follows:

- **Outlier-clustered tiling strategy** that regulates memory access and reduces inefficient workloads. This method overcomes element bit-width variation by clustering all outliers in a single input tiling block, ultimately enhancing data movement and computation efficiency.
- **Hybrid-strategy quantization and reconfigurable double-bit CIM macro array** are designed to address low CIM storage utilization and high latency caused by outlier-based LLM quantization.
- **Quantization factor post-processing strategy and dedicated quantizer** are designed to efficiently perform the multiplication and accumulation of outlier-caused FP-INT workloads with low data bit-width.
- **OutlierCIM** is evaluated at comprehensive benchmarks and achieves up to 4.54× energy efficiency improvement

*The corresponding author: Bo Liu, liubo_cnasic@seu.edu.cn

This work is supported by the National Key Research and Development Program of China under Grant 2023YFB4403103.

and $3.91\times$ speedup compared to baselines. To our best knowledge, **OutlierCIM** is the first work to address LLMs acceleration with CIM-based accelerator and outlier-aware quantization algorithm co-design.

II. BACKGROUND AND MOTIVATION

A. Related Works

Numerous quantization algorithms have been proposed to reduce the memory and computational costs of LLMs by addressing activation outliers [6]–[10]. These methods observe that outliers cluster in specific channels and transfer their extreme magnitudes to weights. However, achieving low bit-width quantization requires complex matrix scaling and transformations [8], [9], which results in low efficiency on general computing platforms. To address this issue, co-design frameworks for digital accelerators and mixed-precision quantization algorithms have been proposed [12]–[15]. These frameworks encode or quantize outliers using large bit-width, enabling near 4-bit LLM inference with dedicated accelerators. However, the balance between hardware and algorithm remains suboptimal. For example, GOBO [12] and OLAccel [13] process outlier-based activation matrices sparsely, leading to complex control logic and irregular memory access. OliVe [14] solves this issue by eliminating neighboring normal values of outliers, but this approach sacrifices model accuracy and introduces element bit-width variation within a single input tiling block. In contrast, Oltron [15] enhances the precision of normal values in outlier channels and regulates memory access through channel-wise permuting. Nevertheless, it encounters inefficiency at post-nonlinear workloads, as only a portion of its processing element (PE) array is capable of FP processing. Moreover, the memory-bound nature of LLMs further restricts the acceleration efficiency of these approaches. Compute-in-memory (CIM) has emerged as a promising solution for memory-bound applications [18]–[20]. However, existing CIM-based accelerators are not optimal for mixed-precision workloads in LLMs, leading to low hardware efficiency.

B. Challenges and Motivation

OutlierCIM seeks to enhance hardware efficiency through a CIM-based accelerator and balance model accuracy with specialized algorithmic strategies. However, several challenges remain to be addressed, as depicted in Fig. 1.

Firstly, large bit-width outliers result in irregular memory access and reduced CIM utilization. On one hand, a significant memory bandwidth is required to maintain a consistent input size. On the other hand, the computation of normal elements must wait additional cycles for outliers due to the bit-serial input nature of the CIM macro. In the case of LLaMA-7B [26] on the WikiText2 [29] dataset, the CIM utilization drops to 58%, and memory energy consumption increases by 23.1%.

Secondly, the outlier-based quantization strategies of current advanced algorithms lead to a conflict between model accuracy and hardware efficiency when processing normal value workloads [7]–[9]. A given 4-bit asymmetric quantized element should be aligned with zero point (ZP) before the

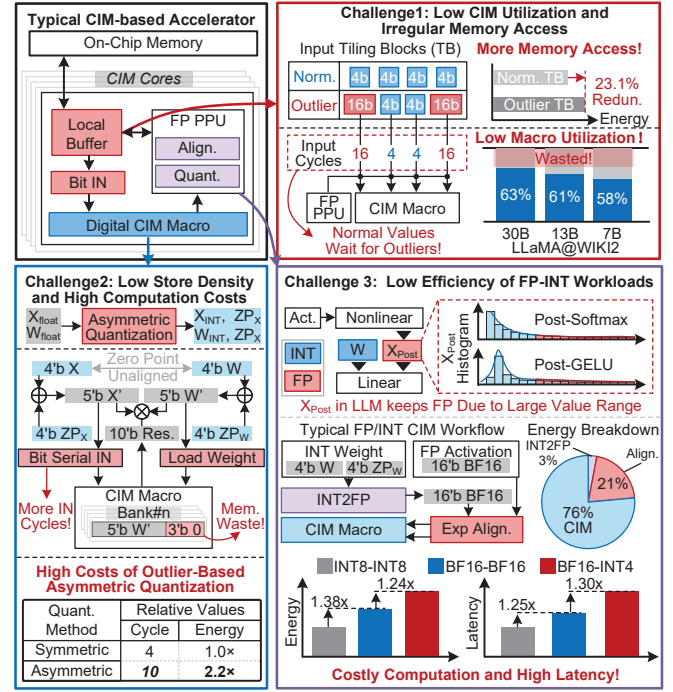


Fig. 1. Typical CIM-based FP/INT accelerator and challenges caused by activation outliers in LLMs

multiplication and accumulation (MAC). This operation leads to high input latency and wastes up to 50% storage density due to the expanded bit-width. Although symmetric quantization introduces lower overheads, it fails to maintain model accuracy at low-bit quantization [15].

Thirdly, outlier-caused FP-INT workloads incur hardware under-efficiency. Given the expanded activation distribution range caused by outliers, the nonlinear layers in LLMs further aggravate this problem. To maintain model accuracy, the activations of post-nonlinear layers should be kept in FP format [22]. Since the missed support of FP-INT computation in current CIM-based accelerators, the INT weights should be converted to FP format and subsequently perform exponent alignment. In the case of LLaMA-7B@WIKI2, this workflow leads to $1.24\times$ energy cost and $1.30\times$ latency.

III. OUTLIERCIM CO-DESIGN FRAMEWORK

The overview of the proposed OutlierCIM, a comprehensive CIM-based accelerator and algorithm co-design framework, is presented in Fig. 2. The structure of this paper is organized as follows. Sec. III-A introduces the outlier-clustered tiling strategy, which regulates memory access and reduces outlier workloads. Sec. III-B introduces the hybrid-strategy quantization and reconfigurable double-bit CIM macro array, which are designed to overcome low storage utilization and high latency. Sec. III-C introduces the quantization factor post-processing strategy and dedicated quantizer, which are designed to boost the calculation efficiency of FP-INT workloads.

A. Outlier-Clustered Tiling Strategy

Given the large dimensions of LLMs, input activations must be loaded and computed in a tiling manner. However, high-

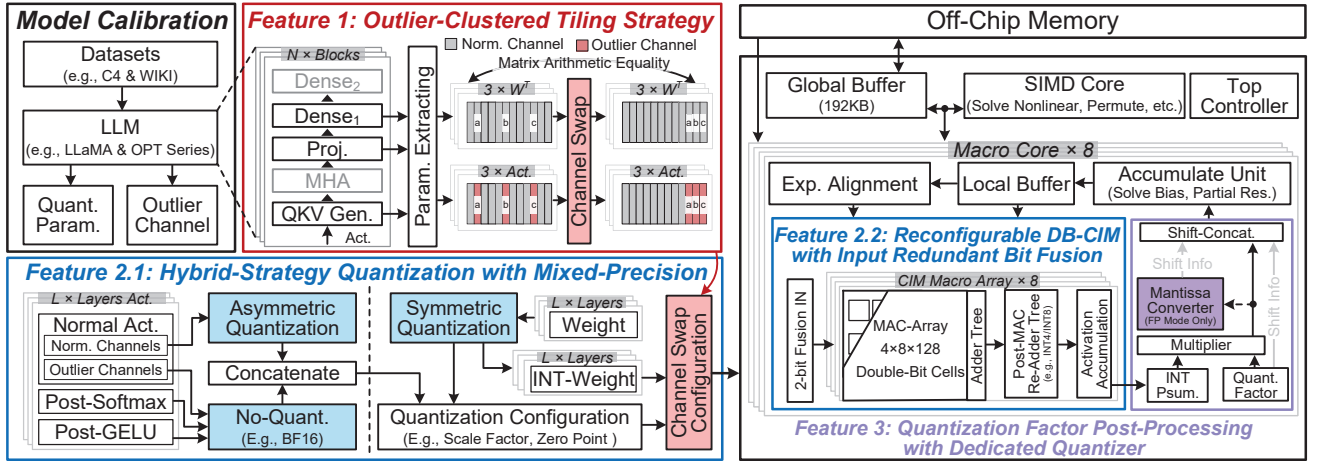


Fig. 2. The overview of proposed algorithm and hardware co-design framework for LLM acceleration

precision outliers significantly increase memory access costs when using the original tiling strategy, illustrated in Fig. 3. OliVe [14] mitigates this issue by nullifying normal values and encoding outliers with large bit-width. However, this tiling strategy is not well-suited for CIM-based architectures. For instance, if a single outlier is quantized using BF16 within a 128-element input vector, the remaining 127 normal values computed in 4 cycles must wait for additional cycles to complete the processing of the current vector.

To address this problem, the Outlier-Clustered Tiling Strategy (OCTS) is proposed, as illustrated in Fig. 3. First, OCTS increases the data precision of normal values within outlier channels. Second, OCTS clusters all outlier channels into the last tiling block of a given activation matrix by swapping outlier channels with the last normal channels. The overall workflow of the proposed OCTS is detailed in Fig. 2 (Feature 1). Outlier channel positions are generated through the model calibration process with the empirical 3σ rule [14]. The activations of the second layer of the Multi-Layer Perceptron (MLP) and the Multi-Head Attention (MHA) layer are not calibrated due to the FP format of post-nonlinear activations [10]. Each tiling block consists of 128 channels, which is sufficiently large to accommodate all outlier channels. As the number of outlier channels may not equal 128, we increase all other normal channels' precision within the last tiling blocks to maintain regular memory access and CIM utilization. Although this strategy introduces additional costs, the small proportion of these last tiling blocks does not significantly affect overall performance (e.g., 128/4096@LLaMA-7B). Finally, the weight dimensions are permuted to ensure the correctness of the general matrix multiplication (GEMM).

The primary advantages of OCTS stem from two key aspects. First, memory access becomes regular compared to the original pattern. Second, low-utilization workloads are eliminated and replaced with uniform data-type tiling blocks. Fig. 4 presents the evaluation of OCTS. The baseline here is a rebuilt bit-serial CIM macro with the original input strategy. OliVe and OCTS tiling strategies are then deployed on the baseline hardware. For OliVe, it fails to reduce latency due

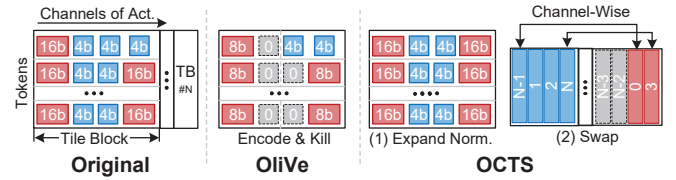


Fig. 3. Comparison of proposed OCTS and other strategies

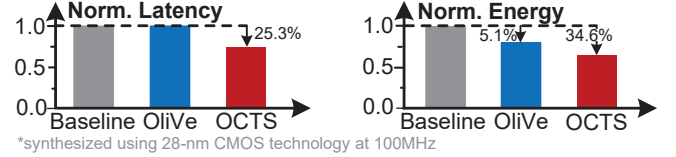


Fig. 4. Performance comparison of outlier-clustered tiling strategy and previous works. The model and dataset here are LLaMA-7B and WikiText2

to low CIM utilization caused by variable element bit-width. However, it achieves energy savings from its regular memory access. In contrast, the proposed OCTS reduces latency by 25.3% and achieves up to 34.6% energy savings compared to the baseline. Nevertheless, these improvements are not ideal due to the inefficient INT-INT and FP-INT workloads, which are further addressed in the following sections.

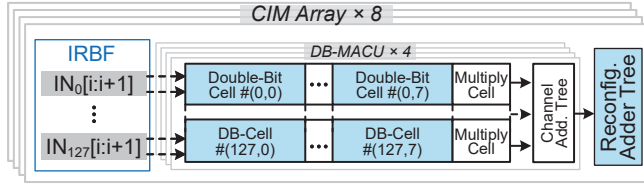
B. Hybrid-Strategy Quantization and Reconfigurable Double-Bit CIM macro array

To address the CIM-unfriendly quantization in LLMs, OutlierCIM introduces a Hybrid-Strategy Quantization (HSQ) depicted in Fig. 2 (Feature 2.1). At the weight aspect, we deploy symmetric quantization, which does not require zero point alignment. The quantization and requantization functions are defined as g_w and g_w^{-1} in Eq. 1, where W is the original FP weight matrix, Q_W is the quantized weight, and Δ_W is the scale factor.

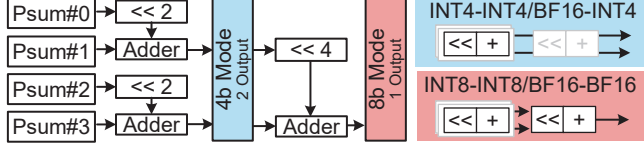
$$Q_W = g_w(W) = \text{round}\left(\frac{W}{\Delta_W}\right) \quad (1)$$

$$W = g_w^{-1}(Q_W) = \Delta_W \times Q_W,$$

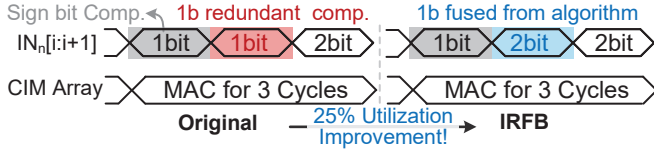
The CIM storage density is therefore improved at $2\times$. Additionally, the performance loss of symmetric quantization is



(a) Proposed double-bit CIM array architecture



(b) Post-MAC reconfigurable adder tree



(c) Comparison of IRFB and original input strategy

Fig. 5. Detailed double-bit CIM array architecture with reconfigurable adder tree (Re-DBCIM) and input redundant bit fusion strategy (IRFB)

compensated by group-wise weight quantization technology co-design, which is then introduced in Sec. III-C. At the activation aspect, asymmetric quantization is still used to maintain the model performance. The quantization and re-quantization functions are defined as f_x and f_x^{-1} , following:

$$Q_X = f_x(X) = \text{round}\left(\frac{X}{\Delta_X}\right) + Z_X \quad (2)$$

$$X = f_x^{-1}(Q_X) = \Delta_X \times (Q_X - Z_X),$$

where X is the original FP activation matrix, Q_X is the quantized activation matrix, Δ_X is the scale factor, and Z_X is the zero point. Notably, the activations of post-nonlinear layers and outlier channels are kept in BF16 format due to their large data distribution range. The quantization scale factors and zero points are generated following the methodology described in OmniQuant [6]. Finally, the quantization configuration is combined with the OCTS strategy to enable regular memory access and high CIM utilization.

Since the expanded bit-width input still introduces high latency, a Double-Bit Reconfigurable CIM macro (Re-DBCIM) array is designed to solve this problem from the hardware aspect. The architecture of Re-DBCIM is shown in Fig. 5 (a), which consists of 4 Double-Bit Cell MAC Units (DB-MACUs). For one DB-MACU, there are 128×8 DB-Cells, which store 2-bit of 128×8 weight matrix. For instance, DB-MACU#0 stores the two most significant bits of a given 128×8 INT4 weight matrix, and DB-MACU#1 stores the remaining two least significant bits. The input 2-bit is then broadcasted to these 2 DB-MACUs, and the MAC operation of each DB-MACU is serially processed by 2-bit multiply-cells and channel-wise adder trees. Subsequently, the MAC results of 2-bit activations and 4-bit weights are generated by weight-wise shift-add. The remaining input bits are then processed similarly and accumulated to operate GEMMs in a tiling manner.

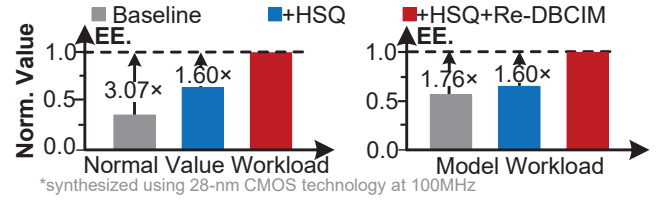


Fig. 6. Energy Efficiency (EE) improvement of gradually deploying proposed algorithm and hardware strategies

To enhance hardware flexibility and expand application scenarios, a reconfigurable adder tree is designed to handle INT8-INT8 or BF16-BF16 workloads, as shown in Fig. 5 (b). At the INT4/BF16-INT4 mode, the first-level shift-add is activated to generate two MAC results as two 128×8 4-bit matrices are stored in the CIM macro array. At the INT8/BF16 mode, all four DB-MACUs store a 128×8 matrix in INT8 or 8b two's complement mantissa format. Subsequently, the second-level shift-add is activated to complete the 8-bit MAC operation, producing one result from the four DB-MACUs.

An Input Redundant Bit Fusion strategy (IRBF) is proposed to address bit-redundancy in the conventional DB-CIM macro [23], as illustrated in Fig. 5 (c). The original DB-CIM macro workflow requires an additional cycle to process the sign bit, resulting in a 25% utilization loss [23]. The IRBF is designed to merge the low-utilization cycle with another bit derived from our proposed hybrid-strategy quantization, where the input is a cardinal bit-width after zero point alignment. Consequently, the utilization of our Re-DBCIM is improved by 25% compared to the conventional DB-CIM.

Fig. 6 illustrates the improvement achieved by gradually deploying the proposed algorithm and hardware strategies. The baseline and model configuration are consistent with Sec. III-A, and OCTS is then deployed for a comprehensive comparison. Compared to the baseline, the HSQ achieves limited energy efficiency improvement due to the high latency caused by expanded input bit-width. The Re-DBCIM is subsequently deployed to address this issue, resulting in a $3.07\times$ energy efficiency improvement at normal value workloads. However, overall model energy efficiency remains constrained by FP-INT workloads, leading to suboptimal results, which is further addressed in Sec. III-C.

C. Quantization Factor Post-Processing Strategy with Dedicated Quantizer

The benefits of quantization strategies are derived from mapping the FP parameters to integers. However, the process of FP-INT workload requires data from these two numerical spaces, resulting in limited efficiency at general computing platforms and conventional FP/INT CIM-based accelerators [20], [21]. The detailed computation process is depicted in Eq. 3, where g_w^{-1} is the requantization function.

$$X_{FP} \times W_{FP} = X_{FP} \times g_w^{-1}(Q_W), \quad (3)$$

The only benefit of the FP-INT process here is low bit-width weight movement. The computation of GEMMs is then operated in the costly FP format.

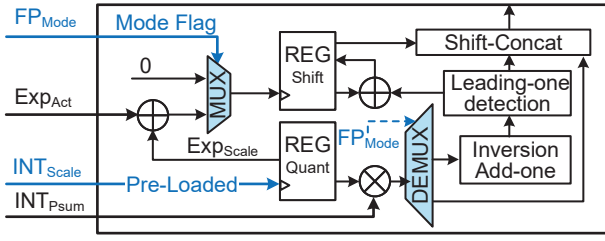


Fig. 7. Proposed quantizer for quantization factor post-processing

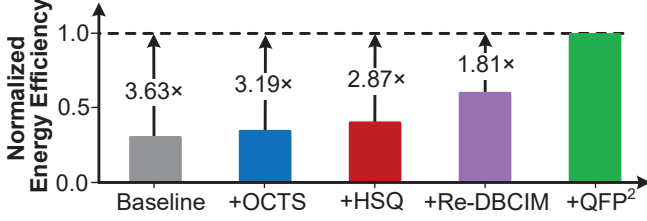


Fig. 8. Improvement breakdown analysis of OutlierCIM

To this end, the Quantization Factor Post-processing (QFP²) strategy is proposed to leverage the benefits of integer data movement and computation. We define that the quantized weights are within the integer numerical space $\mathbb{Z}$, while the original weights exist within the floating-point numerical space $\mathbb{R}$. It is evident that the mathematical spaces $\mathbb{Z}$ and $\mathbb{R}$ are linear homomorphism spaces. Given that the mapping function g_w^{-1} defined in Eq. 1 is a linear function, the linear mapping operation $g_w^{-1} \in \text{Hom}(\mathbb{Z}, \mathbb{R})$ satisfies both the associative and commutative properties [17]. Consequently, the computation of FP-INT workloads is optimized to Eq. 4:

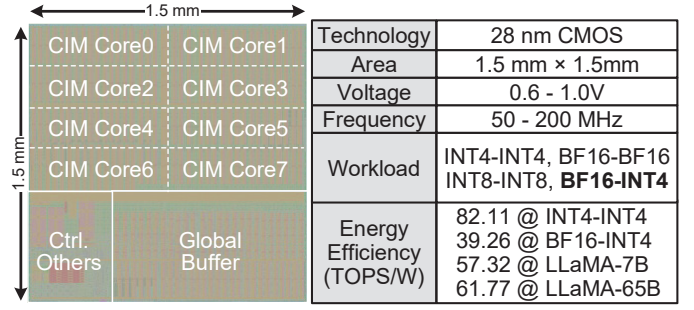
$$X_{FP} \times W_{FP} = g_w^{-1}(X_{FP} \times Q_W), \quad (4)$$

For the MAC operation of FP and INT, the block float point (BFP) computation method is introduced, and the aligned exponent of INT values is set to 0 to match the BFP [20], [23]. For the map function of g_w^{-1} , QFP² follows Eq. 5:

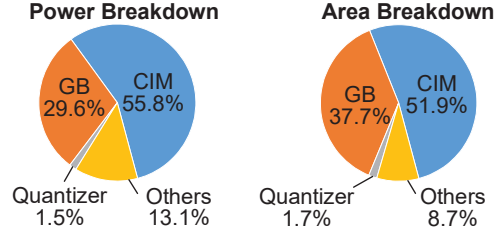
$$\begin{aligned} g_w^{-1}(X_{FP} \times Q_W) &= \Delta_W(X_{FP} \times Q_W) = \frac{A}{2^B}(X_{FP} \times Q_W) \\ &= \frac{A}{2^B} \frac{1}{2^{E_X}}(M_X \times Q_W), \end{aligned} \quad (5)$$

where the phase $\frac{A}{2^B}$ serves as the integer approximation of the scale factor Δ_W , which can be implemented using multiplication and shift operations [24]. Based on the BFP method, E_X , and M_X represent the aligned exponent and two's complement mantissa values of the input X , respectively. Group-wise weight quantization is supported by adjusting Δ_W according to the group size of the weights. Therefore, the MAC operations for FP-INT workloads are efficiently unified and processed in a low bit-width manner.

As shown in Fig. 7, a dedicated quantizer is designed to enhance the efficiency of QFP². INT_{Psum} represents the MAC results of two's complement mantissas M_X and quantized weights Q_W from the CIM macro array. The scale factor Δ_W is decomposed to Exp_{Scale} and INT_{Scale} following Eq. 5. Since the proposed CIM array has an accumulation length of



(a) Layout and summary of OutlierCIM



(b) Power and area breakdown of OutlierCIM

Fig. 9. Layout and breakdown analysis of OutlierCIM accelerator

128, group-wise quantization for 128 elements is inherently supported in an efficient hardware configuration. For the FP mode, the MAC results of $M_X \times Q_W$ are multiplied by the scale factor parameter and subsequently perform leading-one detection for shift configuration. Finally, the aligned input exponent, scale factor parameter, and leading-one information are fused to get the standard IEEE 754 float-point format by configuring the shift-concat module. For the INT mode, only scale factor parameters are used to execute the QFP² and group-wise quantization strategies.

The main benefits of QFP² and quantizer arise from three aspects: 1) the computation of FP-INT workloads is optimized by fully leveraging low bit-width precision; 2) memory access costs are significantly reduced by moving quantized INT4 weights; and 3) model accuracy is maintained by the co-designed group-wise quantization. Fig. 8 demonstrates the improvement breakdown of unified OutlierCIM. The baseline here remains consistent with Sec. III-A. The proposed OutlierCIM achieves an overall 3.63× energy efficiency increase compared to the baseline.

IV. EVALUATION

A. Experimental Setup

Algorithm. The algorithm part of OutlierCIM is evaluated on LLaMA series (7B-65B) [26] and OPT series (6.7B-66B) [27]. The perplexity values of language generation tasks are evaluated on WikiText2 [29] and C4 [28]. For hybrid-strategy quantization and group-wise weight quantization, we reconfigure the OmniQuant [6] to our proposed methods.

Hardware. The OutlierCIM accelerator is implemented using commercial 28nm CMOS technology, occupying an area of 2.25 mm². A cycle-accurate simulator is developed to simulate the performance of the OutlierCIM accelerator, with energy costs derived from the post-layout simulation.

TABLE I
ALGORITHM RESULTS OF OUTLIERCIM AND PREVIOUS WORKS

Model/PPL↓		LLaMA-7B		LLaMA-13B		LLaMA-30B		LLaMA-65B		OPT-6.7B		OPT-13B		OPT-30B		OPT-66B	
Method	A/W bits	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4
FP16	A16 /W16	5.68	7.08	5.09	6.61	4.10	5.98	3.53	5.62	10.86	11.74	10.12	11.19	9.56	10.69	9.34	10.28
OliVe	A4/W4	144.78	117.49	42.24	43.13	36.55	33.78	1.4e7	1.8e7	107.15	61.24	416.57	994.64	334.70	572.02	4058.83	2926.87
SmoothQuant	A4/W4	25.25	32.32	40.05	47.18	192.40	122.38	275.53	244.35	1.8e4	1.5e4	7.4e3	5.6e3	1.2e4	8.3e3	2.2e5	1.8e5
Oltron	A4.1/W4	11.67	15.21	8.20	10.84	6.68	8.65	5.82	8.19	12.00	13.02	11.35	12.27	10.51	11.63	10.49	11.05
OmniQuant	A4/W4	11.26	14.51	10.87	13.78	10.33	12.49	9.17	11.28	12.24	13.56	11.65	13.46	10.60	11.89	10.29	11.35
This Work	A4.1/W4	11.83	15.79	8.63	10.93	6.82	8.74	5.97	8.32	12.17	13.15	11.43	12.35	10.64	11.74	10.52	11.23
	A4.1/W4g128	6.62	8.32	6.49	7.83	5.17	7.25	4.32	6.92	11.32	12.23	11.13	11.97	10.12	11.13	9.92	10.89

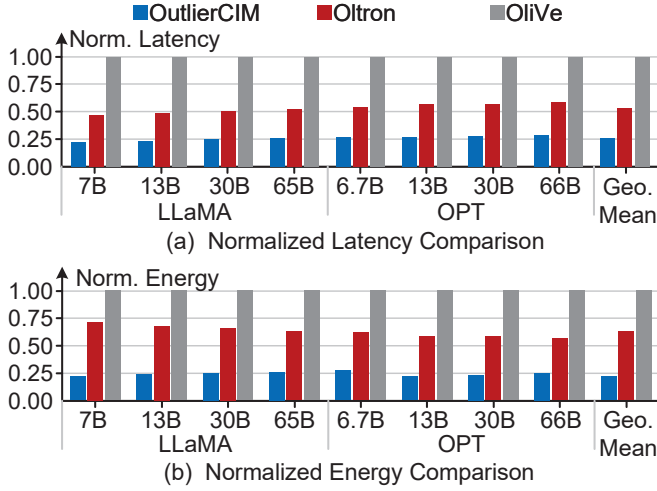


Fig. 10. Comparison of baseline accelerators and OutlierCIM on WikiText2.

Fig. 9 (a) displays the layout and summary of the OutlierCIM accelerator. Notably, the counterpart of the CIM macro array is derived from post-simulation in Virtuoso.

B. Algorithm Result

The comparison between OutlierCIM algorithm and other methods is shown in Table I. The activation bit-width of OutlierCIM is 4.1-bit, as partial activations are stored in BF16 format, following the same definition as Oltron [15]. The perplexity of OutlierCIM without group-wise weight quantization is slightly higher than that of Oltron due to the loss introduced by symmetric weight quantization. However, the co-design framework of OutlierCIM with group-wise weight quantization surpasses the baselines across all benchmarks.

C. Hardware Result

Detailed Configuration. The detailed configuration of OutlierCIM is shown in Fig. 9 (a). The area and power breakdown are presented in Fig. 9 (b).

Performance. For a fair comparison of different architectures, the throughput values of the baseline accelerators are scaled to the same level, following the methodology described in [30]. Fig. 10 (a) highlights the superior performance of OutlierCIM, which achieves an average speedup of $2.06\times$ and $3.91\times$ compared to Oltron and OliVe, respectively.

Energy. Fig. 10 (b) displays the energy comparison between OutlierCIM and the baseline accelerators. OutlierCIM achieves

TABLE II
COMPARISON OF OUTLIERCIM AND PREVIOUS WORKS

	OliVe ISCA'23 [14]	Oltron DAC'24 [15]	ReDCIM JSSC'23 [20]	This Work
Architecture	Digital	Digital	Digital CIM	Digital CIM
Technology	22nm	28nm	28nm	28nm
Voltage	-	-	0.6-1.0V	0.6-1.0V
Frequency	-	-	50-220 MHz	50-220 MHz
Workload	Encoded INT4-INT4	Encoded MFP-INT4 INT4-INT4	FP32-FP32 INT16-INT16 BF16-BF16 INT8-INT8	INT4-INT4 BF16-INT4 BF16-BF16 INT8-INT8
Power	-	-	12.5-69.4 mW	24.5-184.3 mW
Energy Efficiency (TOPS/W)	12.31 ^{*1}	19.32 ^{*1}	36.5@INT8-INT8 ² 9.10@INT16-INT16 29.2@BF16-BF16 3.70@FP32-FP32	82.11@INT4-INT4 39.26@BF16-INT4 57.32@LLaMA-7B 61.77@LLaMA-65B

*1: Results are derived from respective papers and work [16].

*2: Results are quoted from its paper which include sparsity benefits.

lower energy consumption than OliVe and Oltron while delivering higher performance. Specifically, OutlierCIM achieves an average energy efficiency improvement of $4.54\times$ and $2.83\times$ compared to OliVe and Oltron, respectively.

Comparison with State-of-the-Art Works. Table II provides a detailed comparison with state-of-the-art works. ReDCIM [20] is a unified FP/INT CIM-based accelerator, while OliVe [14] and Oltron [15] are digital outlier-aware accelerators. Compared to the baseline CIM-based accelerator, OutlierCIM supports additional computation paradigms and achieves superior energy efficiency for LLM workloads. Moreover, OutlierCIM demonstrates top-tier energy efficiency compared to the baseline outlier-aware accelerators.

V. CONCLUSION

This paper proposes OutlierCIM, the first framework to address LLMs acceleration with a CIM-based accelerator and outlier-aware quantization algorithm co-design. The key insights of OutlierCIM are: 1) At the hardware aspect, a reconfigurable double-bit CIM array is designed to overcome the low efficiency of previous accelerators. 2) At the algorithm aspect, a hybrid-strategy quantization algorithm and a map strategy are proposed to boost the efficiency of the CIM array at workloads caused by outliers. Combining these two parts, the proposed OutlierCIM achieves up to $4.54\times$ energy efficiency and $3.91\times$ speedup compared to baseline accelerators at comprehensive benchmarks.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *Proceedings of the NAACL-HLT, 2019*, pp. 4171-4186.
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, et al., "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, 2019, p. 9.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, arXiv:1409.0473, [Online]. Available: <https://arxiv.org/abs/1409.0473>
- [4] R. Nallapati, B. Zhou, C. N. dos Santos, C. Gulcehre, and B. Xiang, "Abstractive text summarization using sequence-to-sequence rnns and beyond," 2016, arXiv:1602.06023, [Online]. Available: <https://arxiv.org/abs/1602.06023>
- [5] J. Berant, A. Chou, R. Frostig, and P. Liang, "Semantic parsing on free-base from question-answer pairs," *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, 2013*, pp. 1533-1544.
- [6] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, et al., "OmniQuant: Omnidirectionally calibrated quantization for large language models," 2023, arXiv:2308.13137, [Online]. Available: <https://arxiv.org/abs/2308.13137>
- [7] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," *Proceedings of the 40th International Conference on Machine Learning*, vol. 202, 2023, pp. 38087-38099.
- [8] Y. Ma, H. Li, X. Zheng, F. Ling, X. Xiao, et al., "AffineQuant: Affine transformation quantization for large language models," 2024, arXiv:2403.12544, [Online]. Available: <https://arxiv.org/abs/2403.12544>
- [9] H. Lin, H. Xu, Y. Wu, J. Cui, Y. Zhang, et al., "DuQuant: Distributing outliers via dual transformation makes stronger quantized LLMs," 2024, arXiv:2406.01721, [Online]. Available: <https://arxiv.org/abs/2406.01721>
- [10] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," 2022, arXiv:2208.07339, [Online]. Available: <https://arxiv.org/abs/2208.07339>
- [11] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, et al., "A Survey on efficient inference for large language models," 2024, arXiv:2404.14294, [Online]. Available: <https://arxiv.org/abs/2404.14294>
- [12] A. H. Zadeh, I. Edo, O. M. Awad and A. Moshovos, "GOBO: Quantizing attention-based NLP models for low latency and energy efficient inference", *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2020, pp. 811-824.
- [13] E. Park, D. Kim and S. Yoo, "Energy-efficient neural network accelerator based on outlier-aware low-precision computation", in *Proceedings of the 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 688-698.
- [14] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, et al., "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization", in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023.
- [15] C. Xue, C. Zhang, X. Jiang, Z. Gao, Y. Lin, et al., "Oltron: Software-Hardware Co-design for Outlier-Aware Quantization of LLMs with Inter-/Intra-Layer Adaptation", *61st ACM/IEEE Design Automation Conference (DAC)*, 2024.
- [16] J. Lee, W. Lee, and J. Sim, "Tender: Accelerating large language models via tensor decomposition and runtime requantization," *51st International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1048-1062.
- [17] Greub W-H, "Linear algebra", Springer Science & Business Media, 2012, pp. 16-17.
- [18] J. Chen, F. Tu, K. Shao, F. Tian, X. Huo, et al., "AutoDCIM: An Automated Digital CIM Compiler," *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023.
- [19] X. Xiang, Z. Yue, Y. Li, L. Lv, S. Wei, et al., "Dyn-Bitpool: A Two-sided Sparse CIM Accelerator Featuring a Balanced Workload Scheme and High CIM Macro Utilization", *2024 61th ACM/IEEE Design Automation Conference (DAC)*, 2024.
- [20] F. Tu, Y. Wang, Z. Wu, L. Liang, Y. Ding, et al., "ReDCIM: Reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration", *IEEE J. Solid-State Circuits (JSSC)*, vol. 58, no. 1, 2023, pp. 243-255.
- [21] J. Jang, Y. Kim, J. Lee and J. Kim, "FIGNA: Integer unit-based accelerator design for FP-INT GEMM preserving numerical accuracy", *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 760-773.
- [22] X. Hu, Y. Cheng, D. Yang, Z. Yuan, J. Yu, et al., "I-LLM: Efficient Integer-Only inference for Fully-Quantized Low-Bit large language models," 2024, arXiv:2405.17849, [Online]. Available: <https://arxiv.org/abs/2405.17849>
- [23] A. Guo, C. Xi, F. Dong, X. Pu, D. Li et al., "A 28nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs", *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 128-130.
- [24] Z. Li and Q. Gu, "I-ViT: Integer-only quantization for efficient vision transformer inference," *Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023*, pp. 17065-17075.
- [25] D. Zuras, M. Cowlishaw, A. Aiken, M. Applegate, D. Bailey et al., "IEEE standard for floating-point arithmetic." *IEEE Std 754.2008*, 2008, pp. 1-70.
- [26] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M-A Lachaux et al., "LLaMA: Open and efficient foundation language models," 2023, arXiv:2302.13971, [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [27] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, et al., "OPT: Open Pre-trained transformer language models," 2022, arXiv:2205.01068, [Online]. Available: <https://arxiv.org/abs/2205.01068>
- [28] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of machine learning research*, vol. 21, 2020, pp. 1-67.
- [29] S. Merity, C. Xiong, J. Bradbury, R. Socher, "Pointer sentinel mixture models," arXiv:1609.07843, 2016, [Online]. Available: <https://arxiv.org/abs/1609.07843>.
- [30] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, "DOTA: Detect and omit weak attentions for scalable transformer acceleration," *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022, pp. 14-26.